CS 584 Natural Language Processing

Lecture 8: Post-training and Beyond

Yue Ning
Department of Computer Science
Stevens Institute of Technology

Learning Objectives

- Be able to explain:
  - Learning with context
  - Learning from human preference
  - Learning from verifiable rewards

Larger and larger models

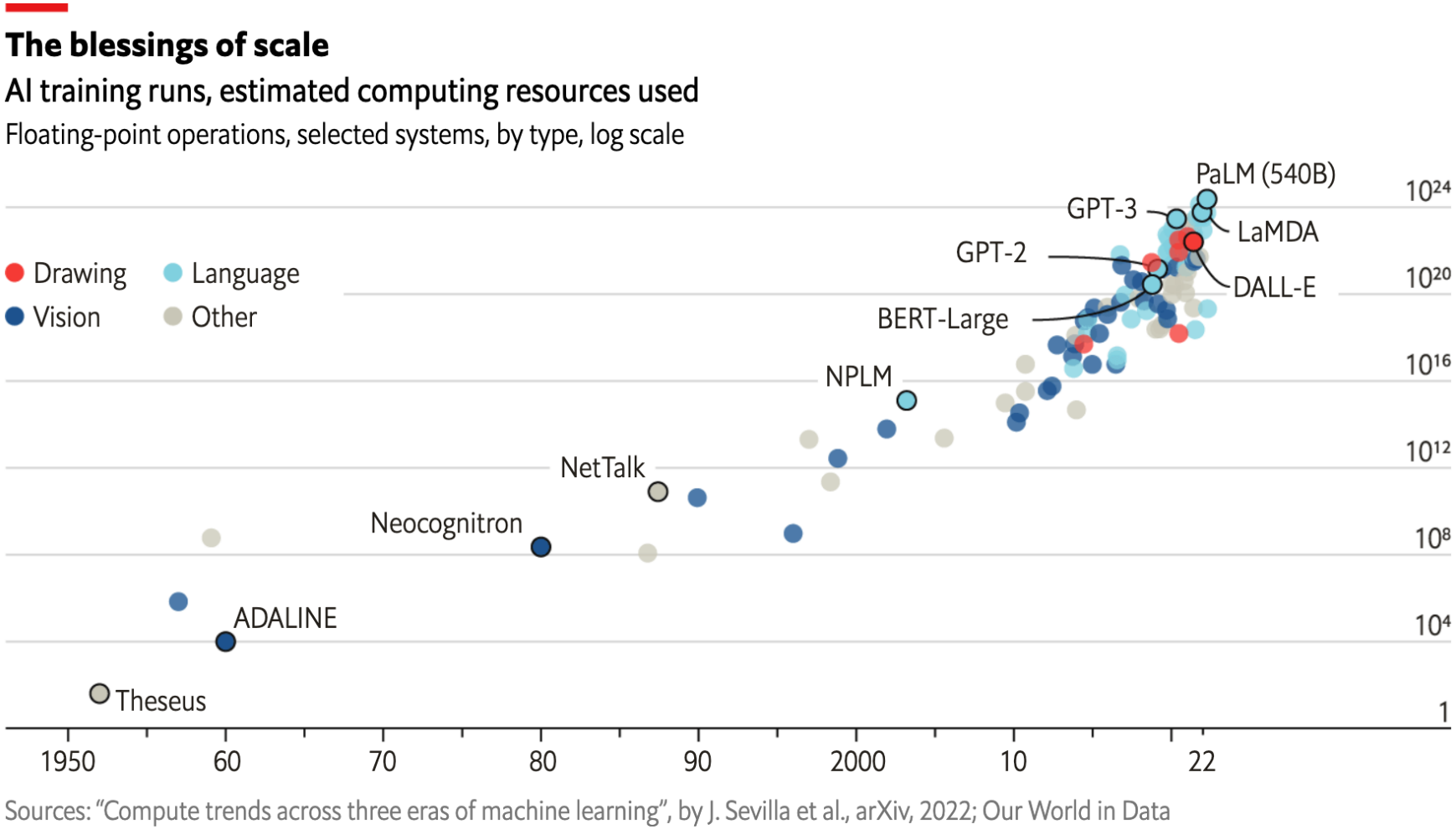
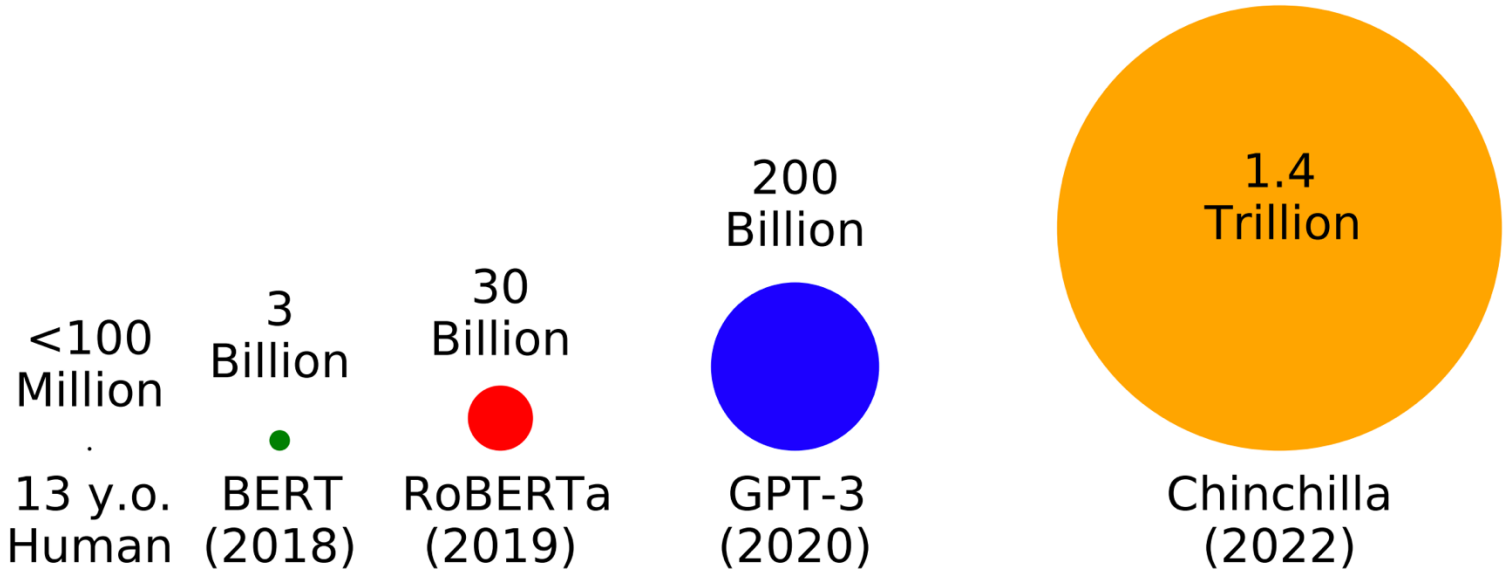


Image source

Trained on more and more data

tokens seen during training

- Huge effort has been put into optimizing LM pretraining at massive scales in the last several years.
- Datasets have also grown by orders of magnitude.



https://babylm.github.io/

Why Pretrained LM and Large LM?

Performance

Context-aware

Abilities

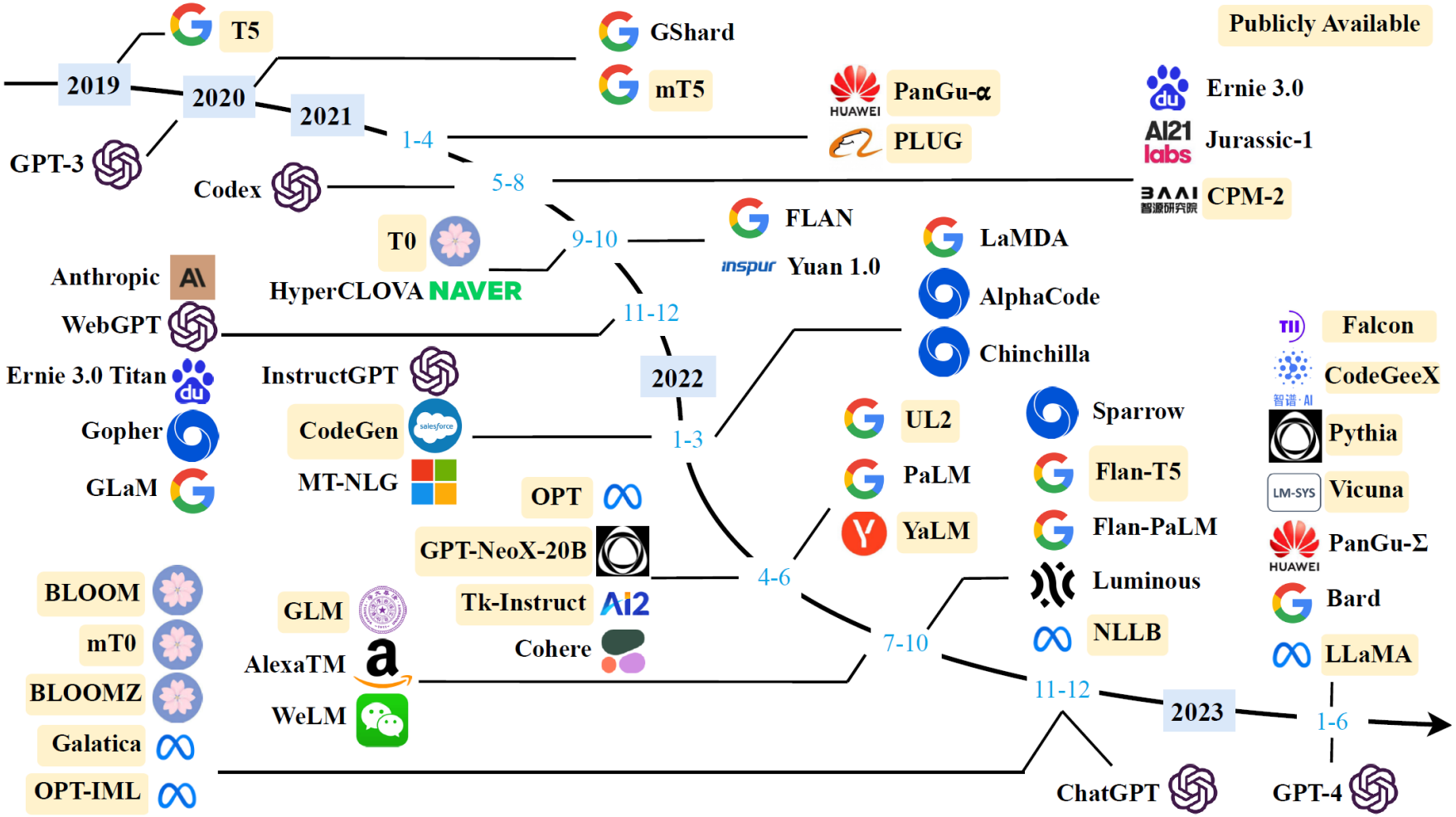
AGI

What kinds of things does pretraining learn?

- Syntax
- Semantics
- Common sense
- Sentiment
- Reasoning
- Basic arithmetic
- And others …

Timeline of existing large language models

Mainly according to the release date of the technical paper for a model.



https://arxiv.org/pdf/2303.18223.pdf

Various Aspects in LLMs

- Model architecture:
  - Encoder-only, Decoder-only, Encoder-Decoder
- Layers components:
  - Tokenization, Normalization, Activation, Attention, …
- Model training:
  - Data preprocessing, Batch training, Learning rate, Optimizer, Human alignment, Scalable training, …

Pre-training Tasks

- Language Modeling (LM):
  - Predicting next token
- Denoising Autoencoding (DAE):
  - Predicting corrupted text
- Mixture-of-Denoisers:
  - LM, DAE with short span, DAE with long span

Part 1: In-context learning / Zero-shot learning

Emergent zero-shot learning

- One key emergent ability in GPT-2 is zero-shot learning:
  - The ability to do many tasks with no examples, and no gradient updates.
- GPT-2 beats SoTA on language modeling benchmarks with no task-specific fine-tuning.
- You can get interesting zero-shot behavior if you're creative enough with how you specify your task!

Heart of the LLMs: In-context learning

- Three approaches to in-context learning:
  - Few-shot
  - One-shot
  - Zero-shot

https://vitalflux.com/in-context-learning-gpt-3-concepts-examples/

New methods of “prompting” LMs

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  cheese =>                           ← prompt
```

Brown et al., 2020

New methods of “prompting” LMs

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

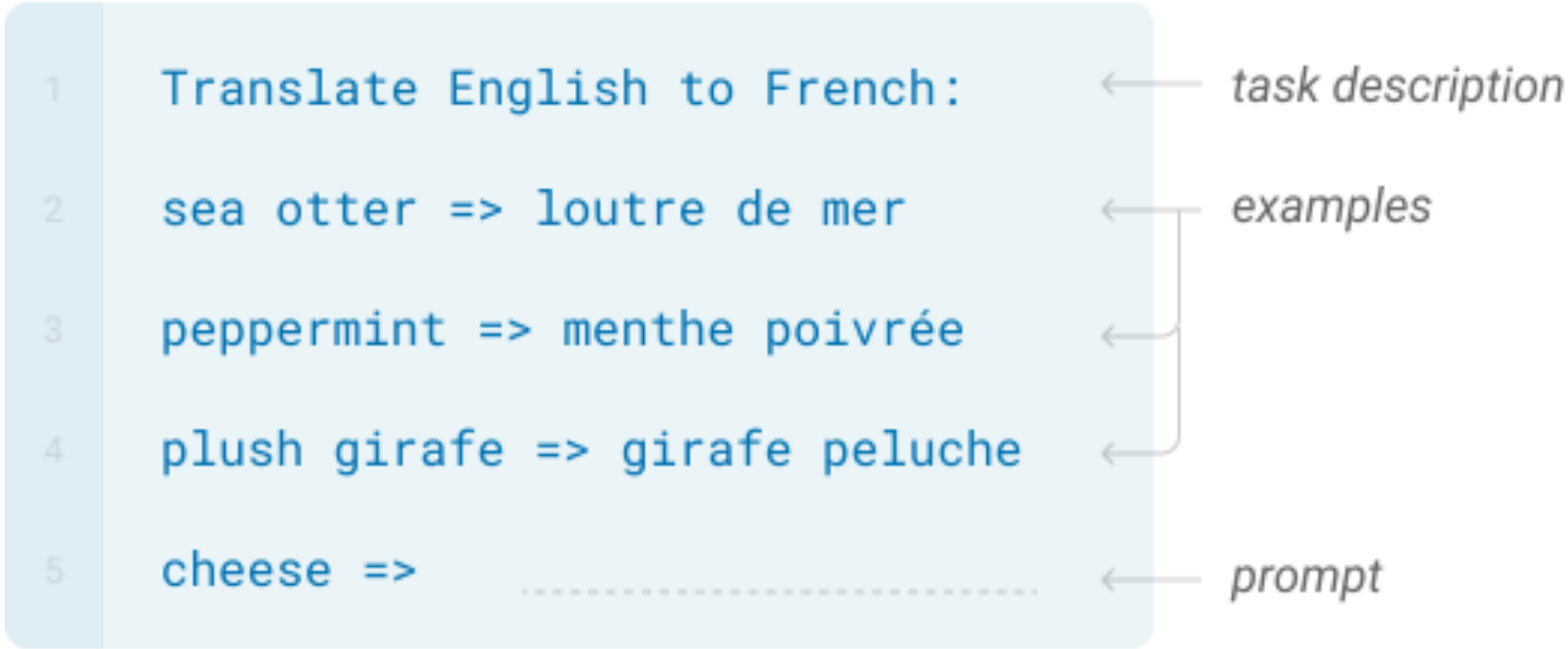
```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← example
3  cheese =>                           ← prompt
```

Brown et al., 2020

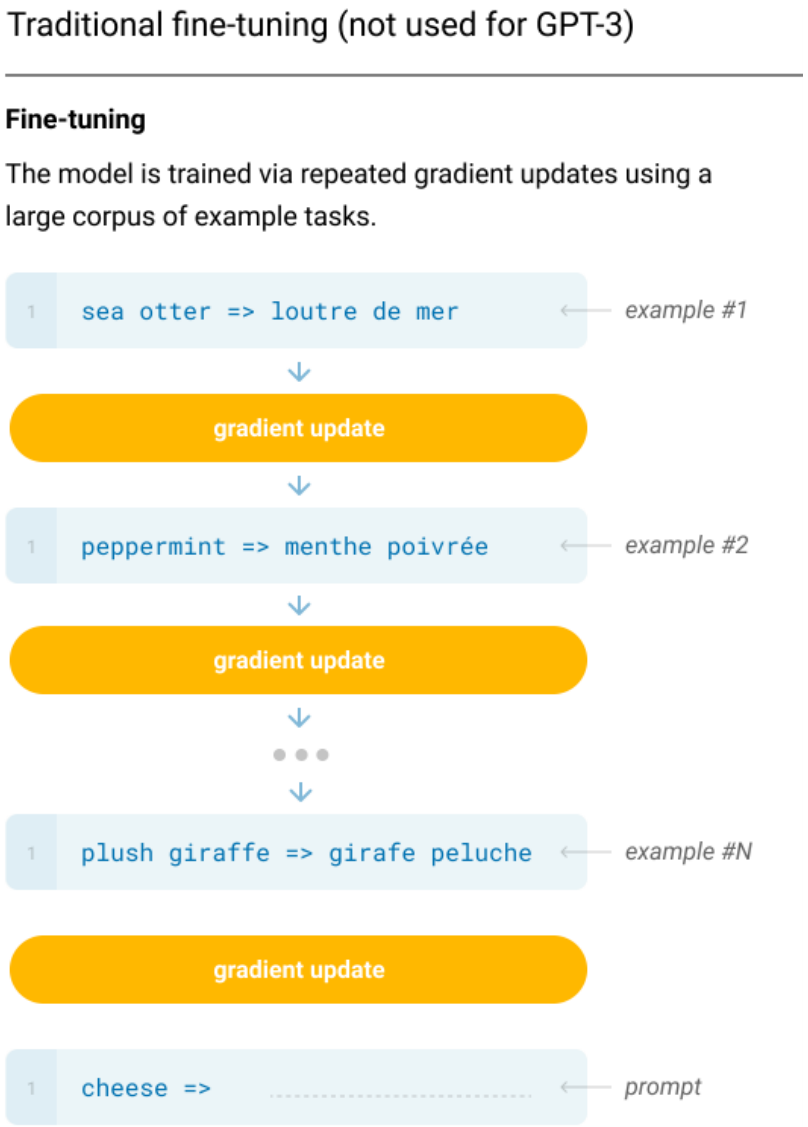
New methods of “prompting” LMs

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



New methods of “prompting” LMs



Brown et al., 2020

Emergent ability on Machine Translation

Zero-shot

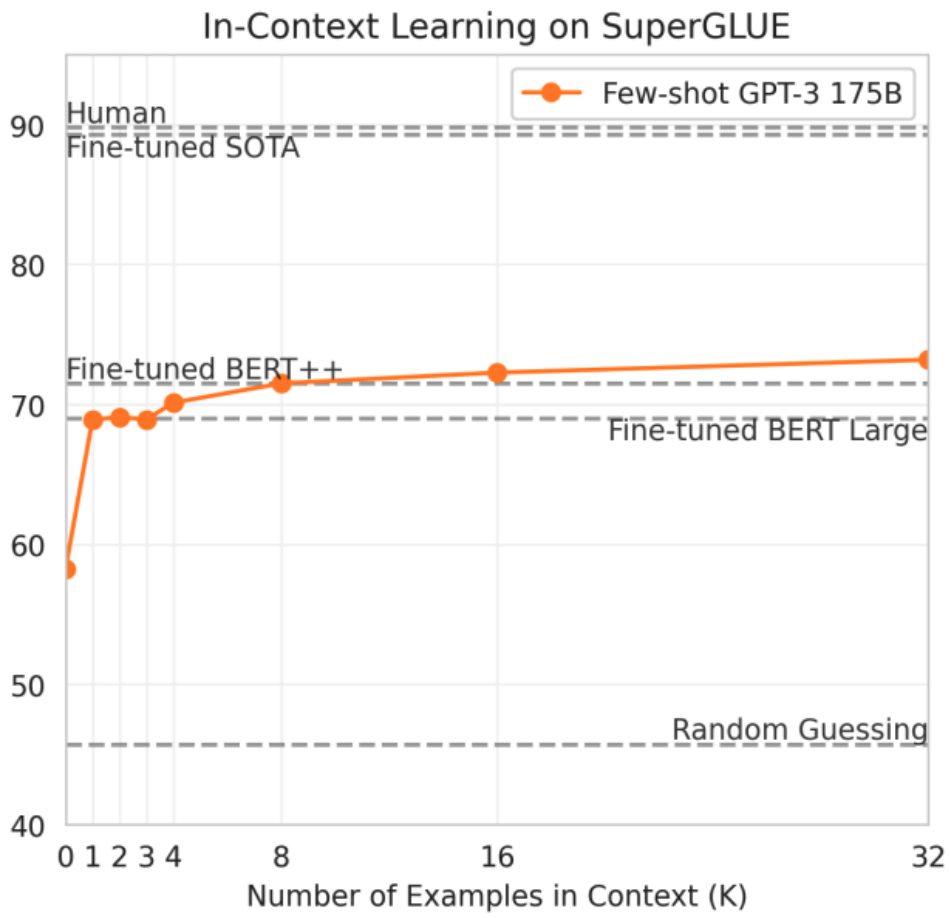
```
1  Translate English to French:
2  cheese =>
```

One-shot

```
1  Translate English to French:
2  sea otter => loutre de mer
3  cheese =>
```

Few-shot

```
1  Translate English to French:
2  sea otter => loutre de mer
3  peppermint => menthe poivrée
4  plush girafe => girafe peluche
5  cheese =>
```



Limits of prompting for harder tasks?

- Some tasks seem too hard for even large LMs to learn through prompting alone.

- Especially tasks involving richer, multi-step reasoning.
- (Humans struggle at these tasks too!)
    - 19583 + 29534 = 49117
    - 98394 + 49384 = 147778
    - 29382 + 12347 = 41729
    - 93847 + 39299 = ?

- Solution: change the prompt!

Prompt Optimization

- A number of methods exist for searching over prompts.
- Most of these do not lead to dramatically better results than doing some manual engineering (and they may be computationally intensive).
- Nevertheless, the choice of prompt is very important in general for zero-shot settings!

Four paradigms in NLP

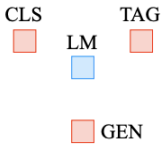
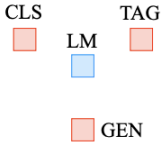
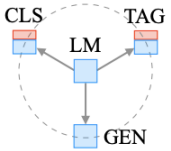
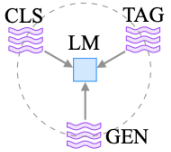
Paradigm	Engineering	Task Relation
a. Fully Supervised Learning (Non-Neural Network)	Features (e.g. word identity, part-of-speech, sentence length)	
b. Fully Supervised Learning (Neural Network)	Architecture (e.g. convolutional, recurrent, self-attentional)	
c. Pre-train, Fine-tune	Objective (e.g. masked language modeling, next sentence prediction)	
d. Pre-train, Prompt, Predict	Prompt (e.g. cloze, prefix)	

Table 1: Four paradigms in NLP. The "**engineering**" column represents the type of engineering to be done to build strong systems. The "**task relation**" column, shows the relationship between language models (LM) and other NLP tasks (CLS: classification, TAG: sequence tagging, GEN: text generation). : fully unsupervised training. : fully supervised training. : Supervised training combined with unsupervised training. indicates a textual prompt. Dashed lines suggest that different tasks can be connected by sharing parameters of pre-trained models. "LM→Task" represents *adapting LMs (objectives) to downstream tasks* while "Task→LM" denotes *adapting downstream tasks (formulations) to LMs*.

Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing

Prompt Types

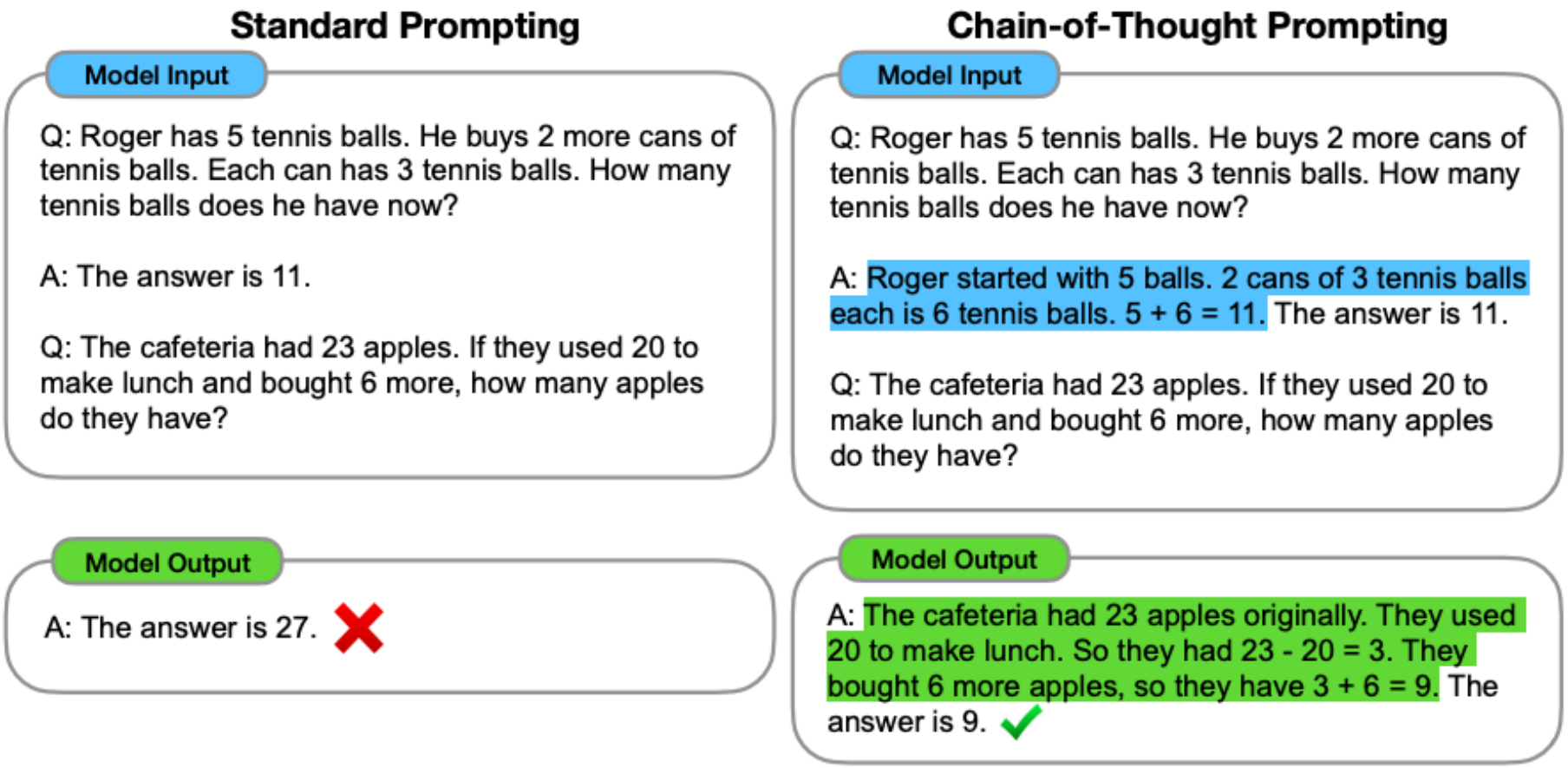
- Cloze prompts: which fill in the blanks of a textual string
  - I love this movie. Overall it was a [Z] movie.
- Prefix prompts: which continue a string prefix.
  - I love this movie. Overall the movie is [Z].

- How to choose: depends on the task and and the model
  - Generation task, or tasks using a standard auto-regressive LM: prefix prompts tend to be more conductive, as they mesh well with the left-to-right nature of the model.
  - Masked LMs: cloze prompts are a good fit, as they very closely match the form of the pre-training task.
  - Full text reconstruction models are more versatile, and can be used with either cloze or prefix prompts.

Chain-of-thought (CoT) prompting

- Chain-of-thought uses natural language as a scaffold for "reasoning"
- Unifies several ideas:
    - For math: relies on the fact that LLMs can do single steps of arithmetic. Builds on that to do multistep problems.
    - For QA: many problems involve reasoning decompositions
        - E.g., What's the capital of the country where Aristotle lived?
        - country = "country where Aristotle lived"
        - return What's the capital of [country]
    - For other tasks: capture the kinds of behavior written in rationales
- Typically, CoT is a few-shot prompting technique, where the in-context examples now contain explanations.
- Answer is not generated in one go, but comes after an explanation that "talks through" the reasoning.

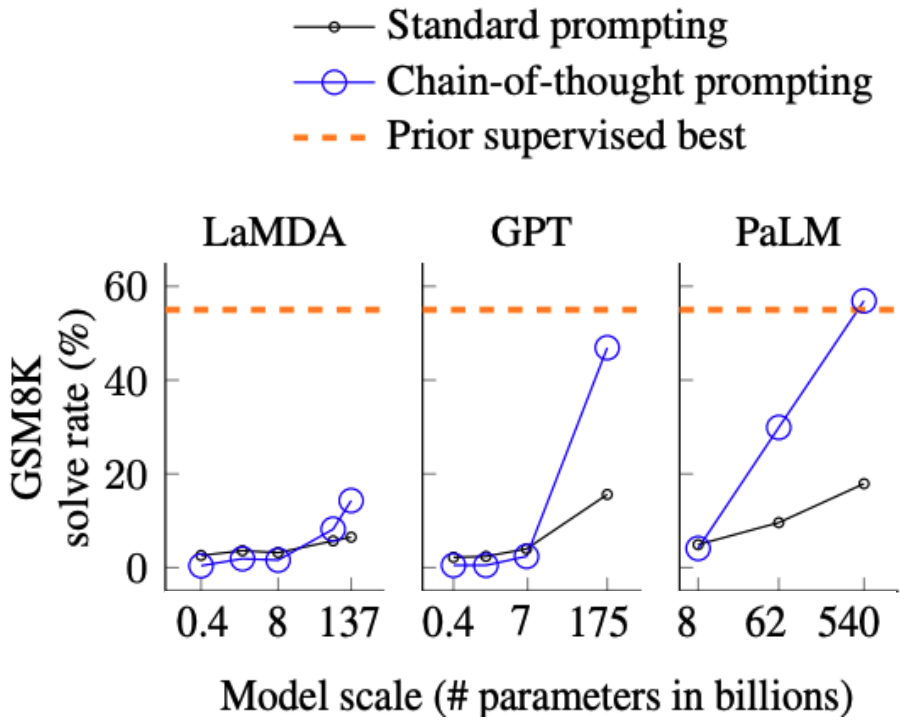
Chain-of-thought prompting



[Wei et al., 2022; also see Nye et al., 2021]

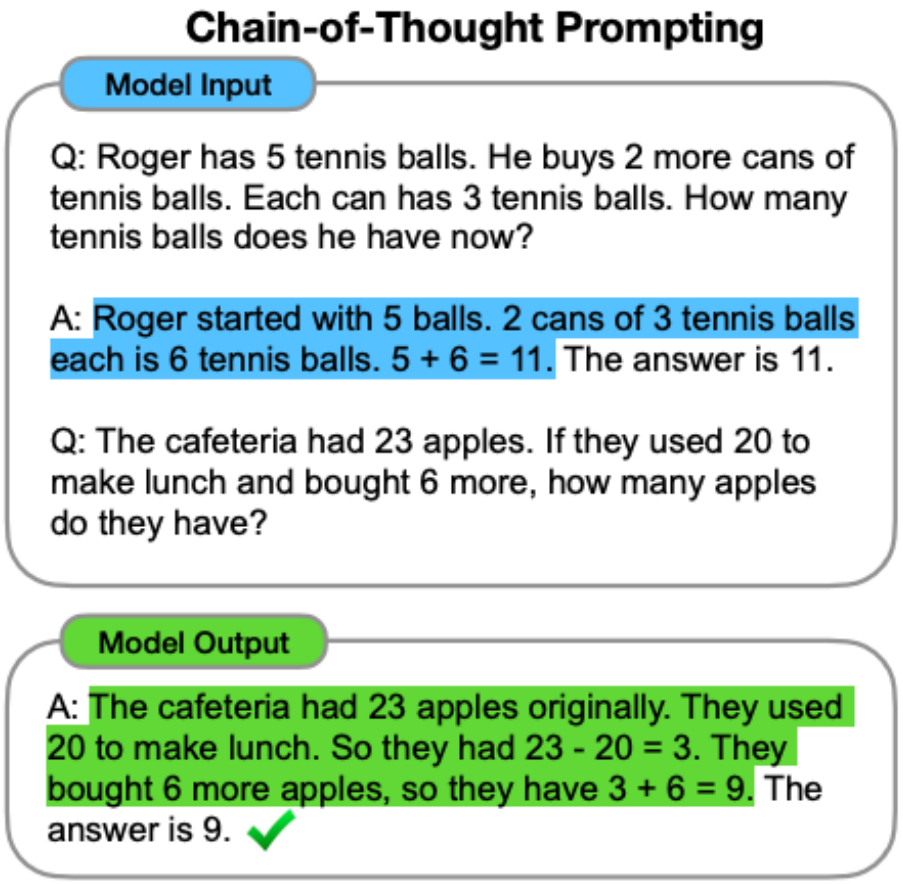
Chain-of-thought prompting performance

- Enables large language models to solve challenging middle school math word problems.
- Notably, chain-of-thought reasoning is an emergent ability of increasing model scale.



[Wei et al., 2022; also see Nye et al., 2021]

Chain-of-thought prompting

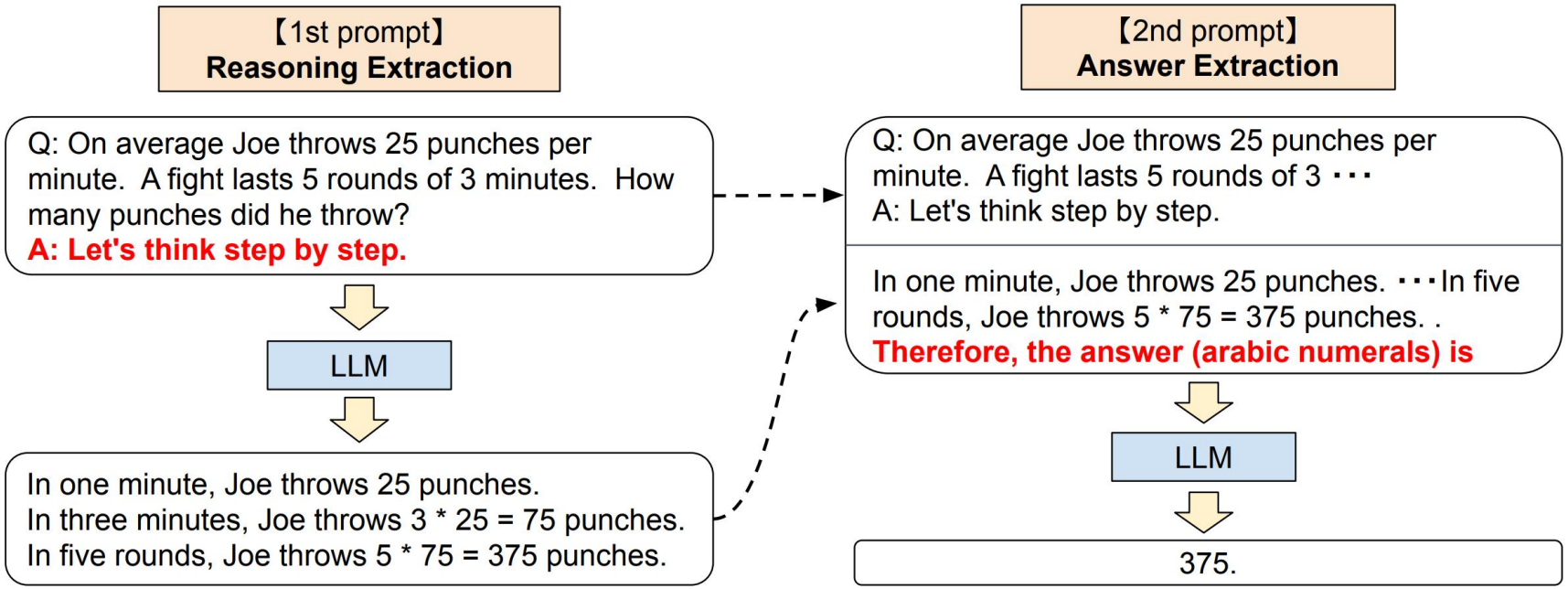


Do we even need examples of reasoning?

Can we just ask the model to reason through things?

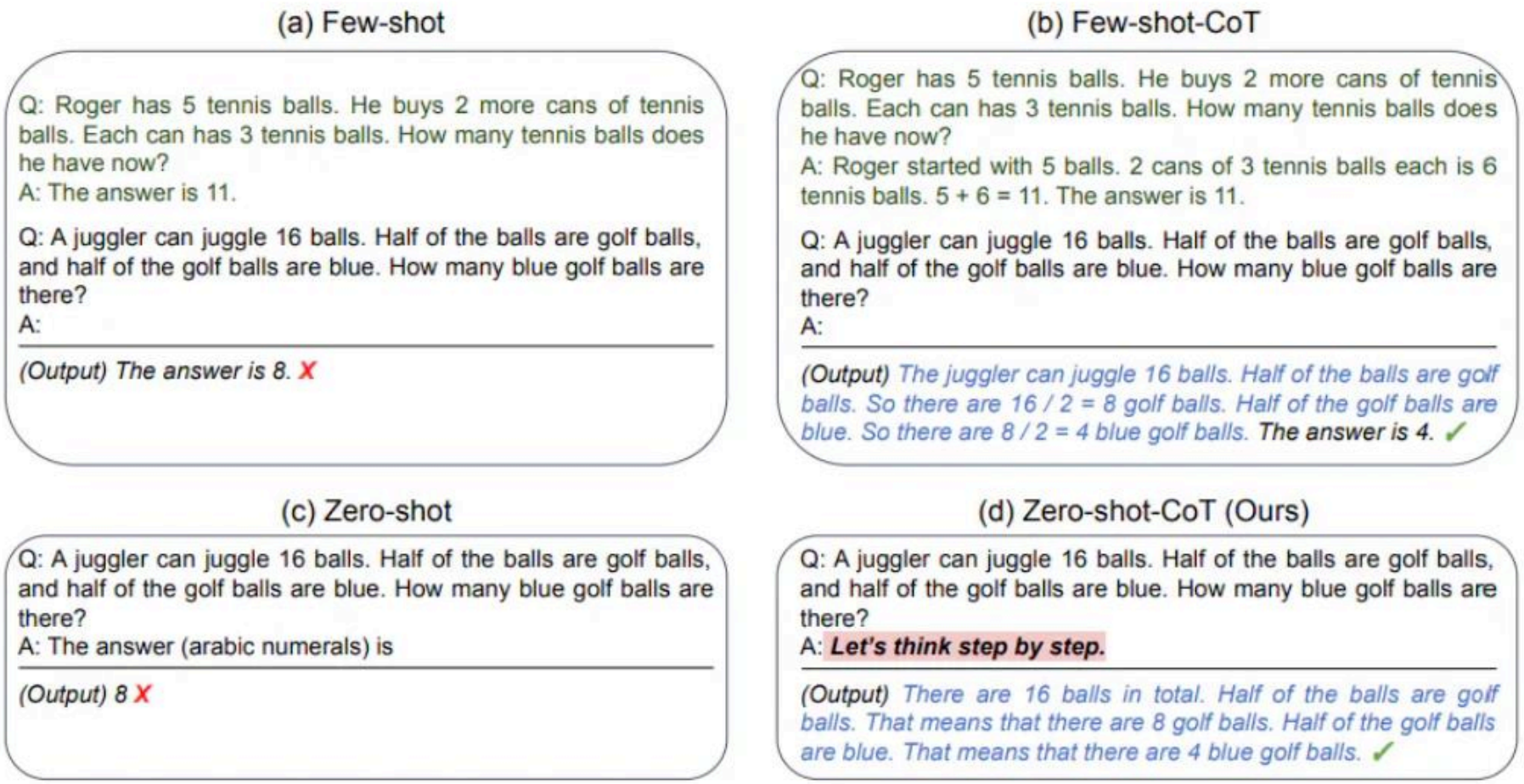
[Wei et al., 2022; also see Nye et al., 2021]

Step-by-Step reasoning



Zero-shot CoT Prompting: step-by-step

Prompt for step-by-step reasoning: produces chains of thought without including demonstrations.



Kojima et al. (2022)

Zero-shot CoT Prompting: step-by-step

	MultiArith	GSM8K
Zero-Shot	**17.7**	**10.4**
Few-Shot (2 samples)	33.7	15.6
Few-Shot (8 samples)	33.8	15.6
Zero-Shot-CoT	**78.7**	**40.7**
Few-Shot-CoT (2 samples)	84.8	41.3
Few-Shot-CoT (4 samples : First) (*1)	89.2	-
Few-Shot-CoT (4 samples : Second) (*1)	90.5	-
Few-Shot-CoT (8 samples)	93.0	48.7

Greatly outperforms zero-shot →

Manual CoT still better →

Kojima et al. (2022)

Zero-shot CoT Prompting: step-by-step

No.	Category	Template	Accuracy
1	instructive	Let's think step by step.	**78.7**
2		First, (*1)	77.3
3		Let's think about this logically.	74.5
4		Let's solve this problem by splitting it into steps. (*2)	72.2
5		Let's be realistic and think step by step.	70.8
6		Let's think like a detective step by step.	70.3
7		Let's think	57.5
8		Before we dive into the answer,	55.7
9		The answer is after the proof.	45.7
10	misleading	Don't think. Just feel.	18.8
11		Let's think step by step but reach an incorrect answer.	18.7
12		Let's count the number of "a" in the question.	16.7
13		By using the fact that the earth is round,	9.3
14	irrelevant	By the way, I found a good restaurant nearby.	17.5
15		Abrakadabra!	15.5
16		It's a beautiful day.	13.1
-		(Zero-shot)	17.7

Prompt engineering: OpenAI’s six strategies

- Write clear instructions.
- Provide reference text.

- Give LLM simpler tasks.
- Give the model time to think.

- Use external tools.
- Test changes systematically.

Summary: In-Context Learning

- Advantages:
  - No finetuning needed
  - Prompt engineering (e.g. CoT) can improve performance
- Limitations:
  - Limits to what you can fit in context
  - Complex tasks will probably need gradient steps

Instruction Finetuning

PROMPT	*Explain the moon landing to a 6 year old in a few sentences.*

COMPLETION	GPT-3
	`Explain the theory of gravity to a 6 year old.`
	`Explain the theory of relativity to a 6 year old in a few sentences.`
	`Explain the big bang theory to a 6 year old.`

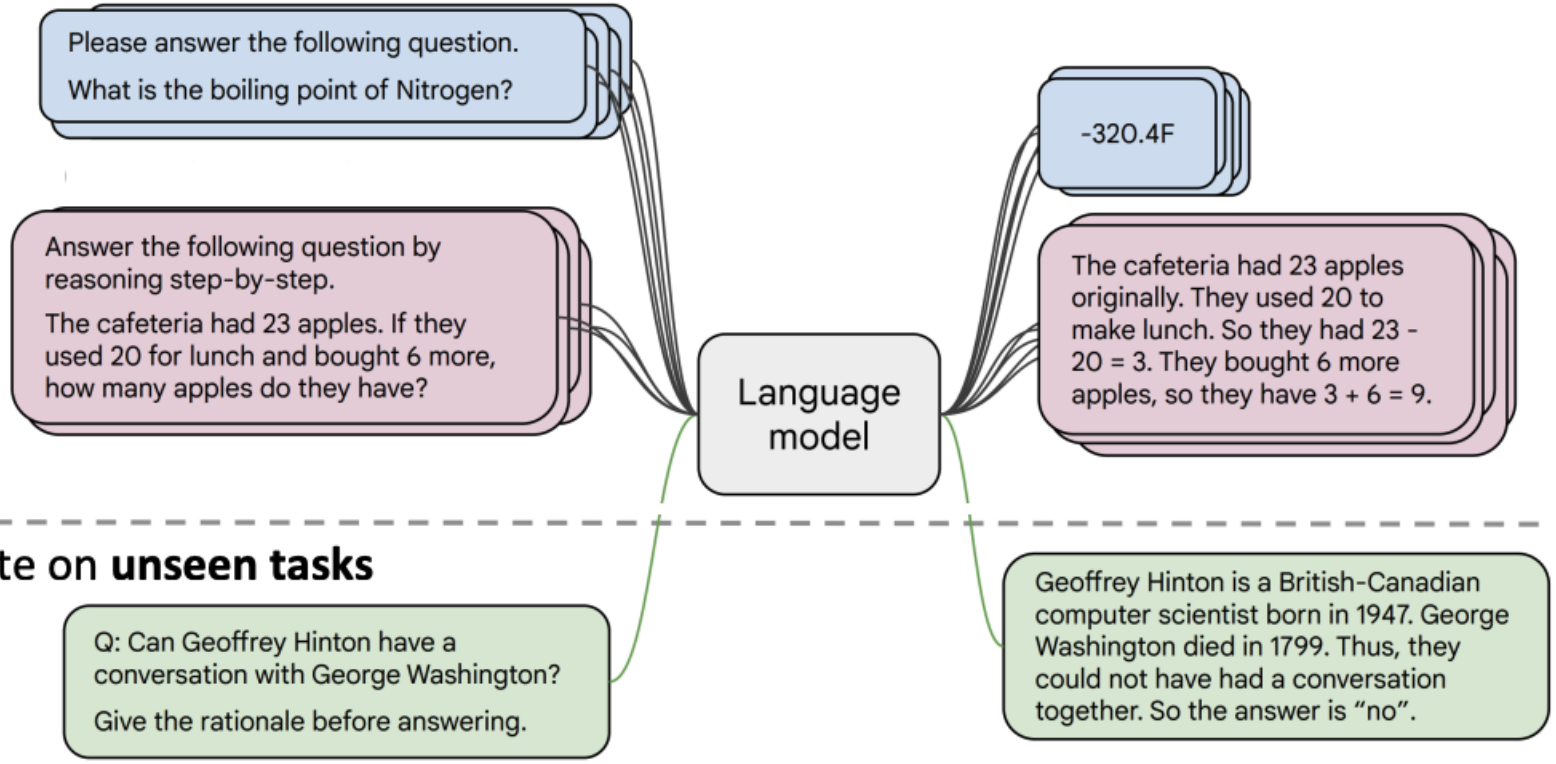
PROMPT	*Explain the moon landing to a 6 year old in a few sentences.*
COMPLETION	**Human** A giant rocket ship blasted off from Earth carrying astronauts to the moon. The astronauts landed their spaceship on the moon and walked around exploring the lunar surface. Then they returned safely back to Earth, bringing home moon rocks to show everyone.

Language models are not aligned with user intent.
Finetuning to the rescue!

[Ouyang et al., 2022]

Instruction Finetuning

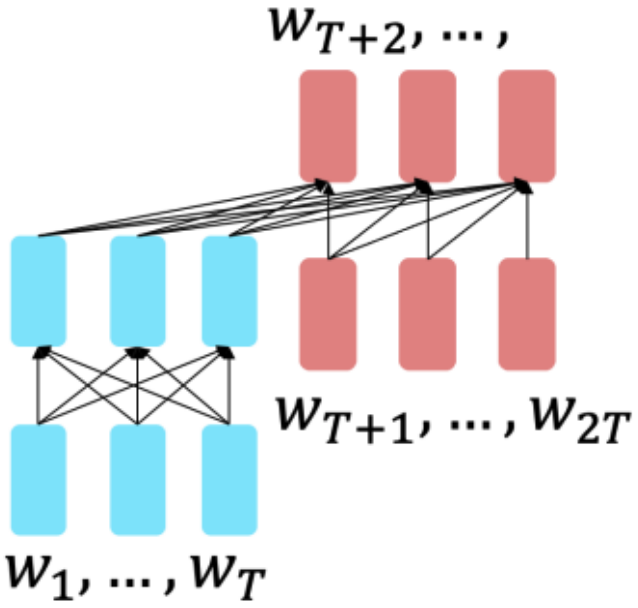
- **Collect examples** of (instruction, output) pairs across many tasks and finetune an LM



[FLAN-T5; Chung et al., 2022]

Instruction Finetuning

- Recall the T5 encoder-decoder model [Raffel et al., 2018], pretrained on the span corruption task
- Flan-T5 [Chung et al., 2020]: T5 models finetuned on 1.8K additional tasks.

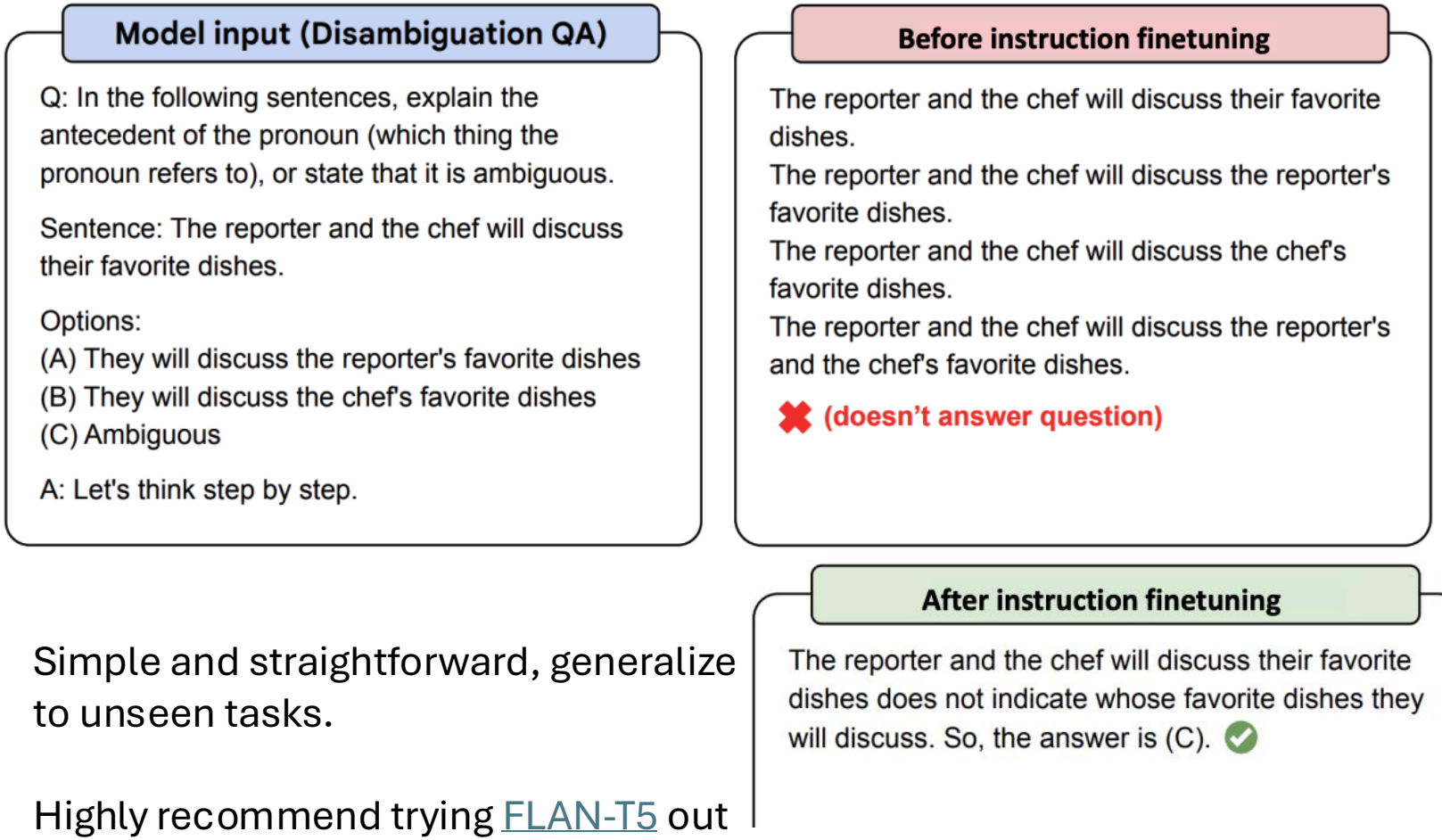


Params	Model	BIG-bench + MMLU avg (normalized)
80M	T5-Small	-9.2
	Flan-T5-Small	-3.1 (+6.1)
250M	T5-Base	-5.1
	Flan-T5-Base	6.5 (+11.6)
780M	T5-Large	-5.0
	Flan-T5-Large	13.8 (+18.8)
3B	T5-XL	-4.1
	Flan-T5-XL	19.1 (+23.2)
11B	T5-XXL	-2.9
	Flan-T5-XXL	23.7 (+26.6)

Bigger model = bigger Δ

[Chung et al., 2022]

Instruction finetuning



- Simple and straightforward, generalize to unseen tasks.
- Highly recommend trying FLAN-T5 out to get a sense of its capabilities.

Limitations of instruction finetuning

- Expensive to collect ground truth data for tasks.
- Open-ended creative generation have no right answer.
  - Write me a story about a dog and her pet grasshopper.
- LM penalizes all token-level mistakes equally, but some errors are worse than others.
  - Such as choosing a completely unrelated word or changing the meaning of a sentence, can be more severe than minor mistakes in word choice.
- Mismatch between the LM objective and the objective of "satisfy human preferences"!

Can we explicitly attempt to satisfy human preferences?

Part 2: Learning from Preference

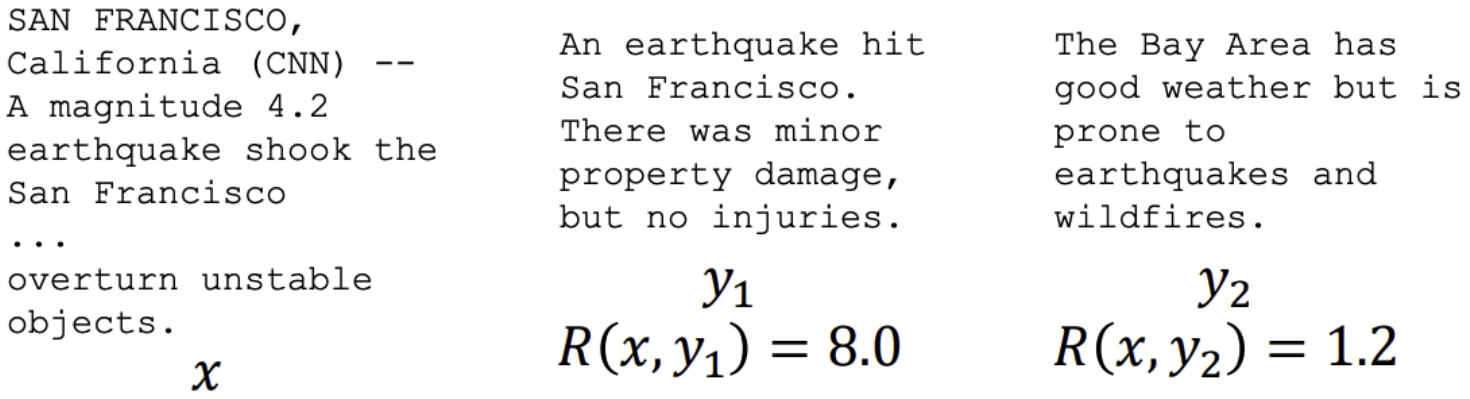
Reinforcement Learning from Human Feedback (RLHF)

- RLHF employs reinforcement learning (RL) to adapt LLMs to human feedback by learning a reward model.
- Three components:
  - A pre-trained LM to be aligned: typically a generative model that is initialized with existing pretrained LM parameters.
  - A reward model learning from human feedback: provides (learned) guidance signals that reflect human preferences for the text generated by the LM, usually in the form of a scalar value.
    - The reward model can take on two forms: a fine-tuned LM or a LM trained using human preference data.
  - A RL algorithm training the LM: Proximal Policy Optimization (PPO) is a widely used.

[Ouyang et al., 2022]

Goal: Optimizing for human preferences

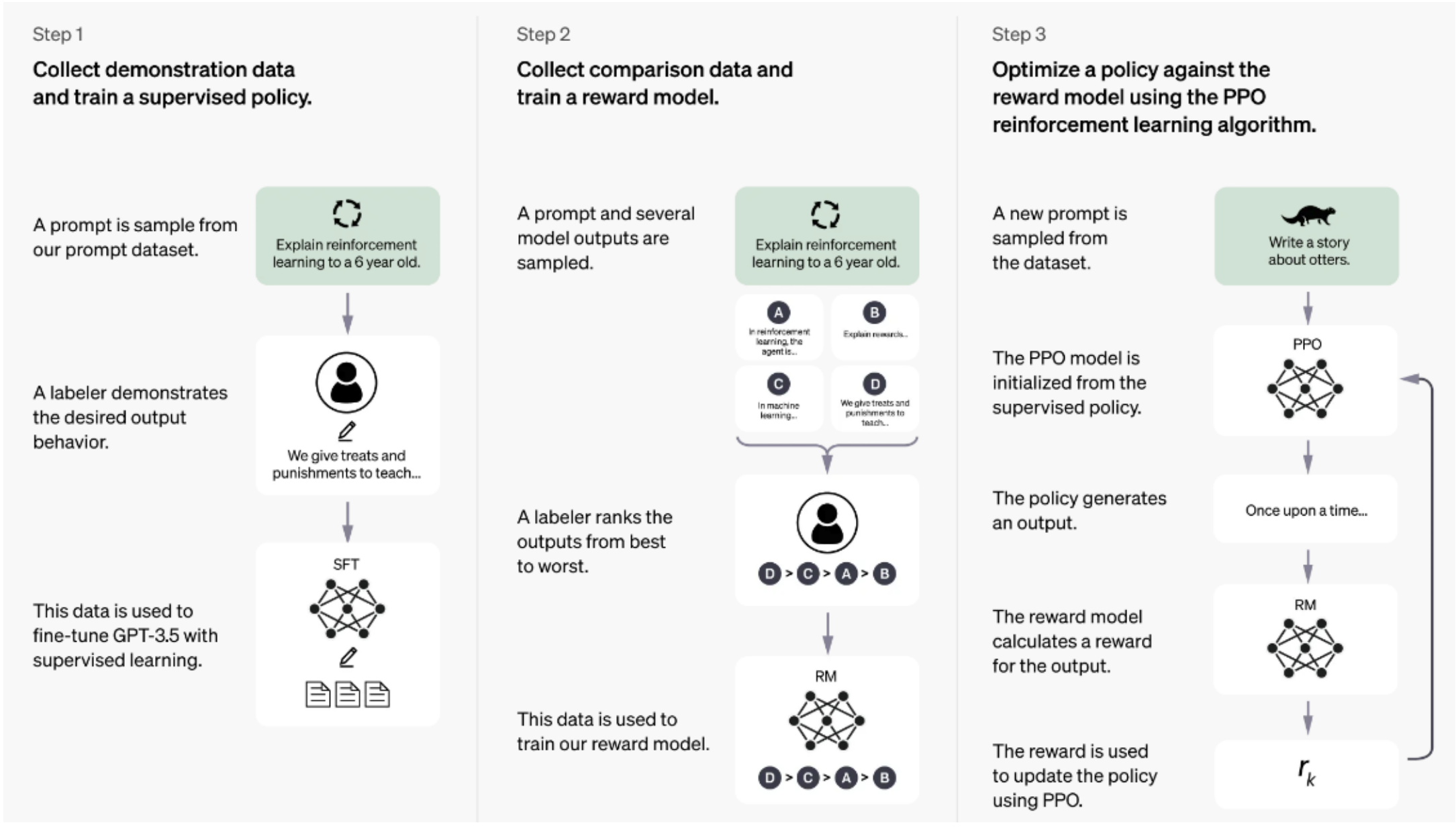
- Let's say we were training a LM on some task (e.g., summary)
- For an instruction x and a LM sample y, it's hard to obtain an absolute "quality rating" $R(x, y)$ like the following:



- We only have preference data from human annotators, in the form of:

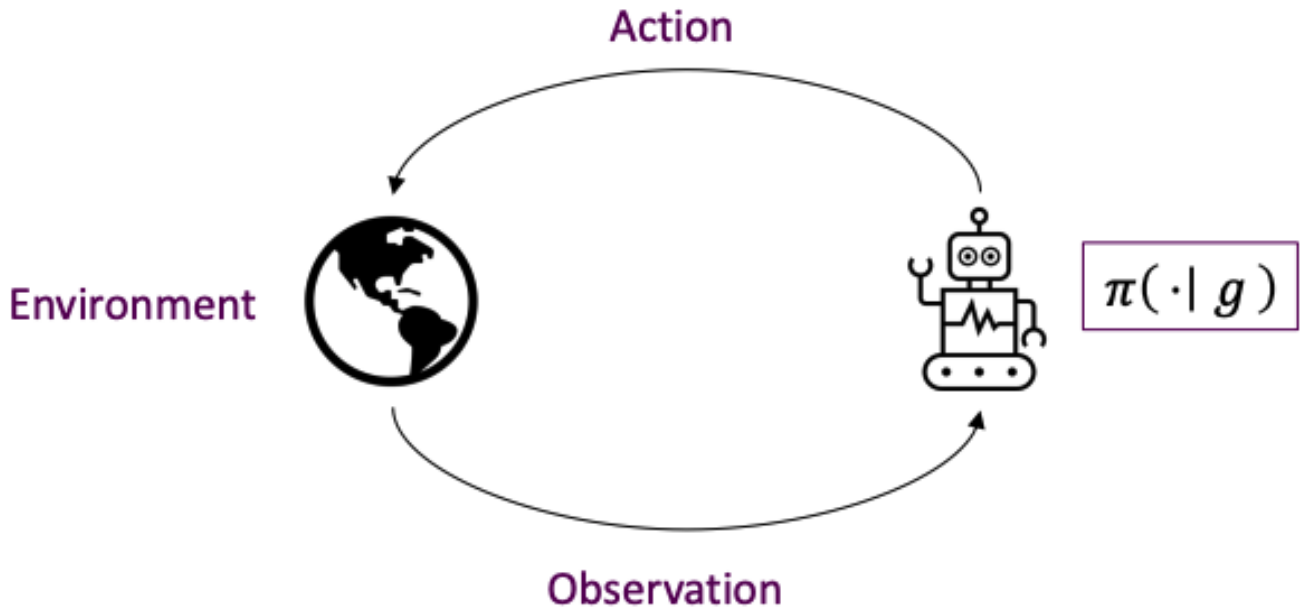
$$R(x, y_1) > R(x, y_2)$$

High-level instantiation: ‘RLHF’ pipeline



Language model is the “policy model”

- In Reinforcement Learning, we optimize a policy model π.
- The policy model decides the next-step actions, based on the observation.
- In RLHF, the language model is the policy model.
- The observation is the input text.



Train a reward model

- Problem: Human-in-the-loop is expensive.
- Solution: Don’t ask for humans for preferences. Instead, model their preferences with a reward model.
- Train a reward model $RM_\phi(x, y)$ that predicts human reward from an annotated dataset.

Preference learning

- This comparison model follows that of Bradley-Terry (1952):

- Train a model RM with parameters ϕ:

$$J = -\mathrm{E}_{(x,y^w,y^l)}[\log \sigma(RM_\phi(x, y^w) - RM_\phi(x, y^l))]$$

- y^w is the "winning" sample
- y^l is the "losing" sample
- σ is the Sigmoid function $\sigma(z) = \frac{1}{1+e^z}$

- Intuition: By minimizing J, we are encouraging $RM_\phi(x, y^w) - RM_\phi(x, y^l)$ to be large.

- We want the model output of the winning sample to be higher than the losing sample.

- The trained model RM the reward model.

Optimize the language model

- Now we have the following:
  - A pretrained (possibly instruction-finetuned) LM $p_\theta^{PT}(y|x)$
  - A reward model $RM_\phi(x, y)$ that produces scalar rewards for LM outputs, trained on a dataset of human comparisons.
- The next step:
  - Optimize θ in $p_\theta^{PT}(y|x)$ into $p_\theta^{RL}(y|x)$ by maximizing: $E_{\hat{y}\sim p_\theta(y|x)}[RM_\phi(x, \hat{y})]$
  - Regularization: Add a KL regularization term to avoid θ from going too far.

$$\min_\theta E_{\hat{y}\sim p_\theta(y|x)}[RM_\phi(x, \hat{y}) - \beta\log(\frac{p_\theta^{RL}(\hat{y}|x)}{p_\theta^{PT}(\hat{y}|x)})]$$

This minimization can use a reinforcement learning algorithm (e.g., Proximal Policy Optimization).

PPO is out of scope for CS584.

InstructGPT

PROMPT *Explain the moon landing to a 6 year old in a few sentences.*

COMPLETION

GPT-3

```
Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

Explain evolution to a 6 year old.
```

InstructGPT

```
People went to the moon, and they took pictures of what they saw, and
sent them back to the earth so we could all see them.
```

ChatGPT: optimizing language for dialogue

- Note: OpenAI (and similar companies) are keeping more details secret about ChatGPT training (including data, training parameters, model size)— perhaps to keep a competitive edge…

Instruction finetuning!

We trained this model using Reinforcement Learning from Human Feedback (RLHF), using the same methods as InstructGPT, but with slight differences in the data collection setup. We trained an initial model using supervised fine-tuning: human AI trainers provided conversations in which they played both sides—the user and an AI assistant. We gave the trainers access to model-written suggestions to help them compose their responses. We mixed this new dialogue dataset with the InstructGPT dataset, which we transformed into a dialogue format.

To create a reward model for reinforcement learning, we needed to collect comparison data, which consisted of two or more model responses ranked by quality. To collect this data, we took conversations that AI trainers had with the chatbot. We randomly selected a model-written message, sampled several alternative completions, and had AI trainers rank them. Using these reward models, we can fine-tune the model using Proximal Policy Optimization. We performed several iterations of this process.

RLHF!

Thoughts of the RLHF pipeline

- Dataset: human preference comparison data
- Goal: train the LM $p_\theta^{PT}(y|x)$ into $p_\theta^{RL}(y|x)$
- Current RLHF solution:
  - First train a reward model $RM_\phi(x, \hat{y})$
  - Then use it to train the LM
- Can we train the LM (with parameters θ) directly from the preference data?
- → Direct Preference Optimization (DPO)!

Direct Preference Optimization (DPO)

- Recall that in RLHF, we trained the RM ϕ to maximize this objective:

$$E_{\hat{y}\sim p_\theta(y|x)}\, J$$

where:

$$J = RM_\phi(x, \hat{y}) - \beta \log(\frac{p_\theta^{RL}(\hat{y}|x)}{p_\theta^{PT}(\hat{y}|x)})]$$

- Suppose there's a minimum value J^*, then J^* only depends on x and is constant to ϕ.
- We can rearrange the terms into:

$$RM_\phi(x, \hat{y}) = \beta \log \frac{p^{RL}(\hat{y}|x)}{p^{PT}(\hat{y}|x)} + J^*$$

- Regardless of ϕ or x or y, the value of J^* does not change.
  - So, in case there is the equation with the format $RM_\phi(x, y^w) - RM_\phi(x, y^l)$, we can cancel out J^*

Rafailov et al (2023)

Direct Preference Optimization (DPO)

- Recall, how we fit the reward model $RM_\phi(x, \hat{y})$:

$$J_{RM}(\phi) = -E_{(x,y^w,y^l)\sim D}[\log \sigma(RM_\phi(x, y^w) - RM_\phi(x, y^l))]$$

- We can plug in the expressions of RM computing the rewards for y^w and y^l:

$$RM_\phi(x, y^w) - RM_\phi(x, y^l) = \beta \log \frac{p_\theta^{RL}(y^w|x)}{p^{PT}(y^w|x)} - \beta \log \frac{p_\theta^{RL}(y^l|x)}{p^{PT}(y^l|x)}$$

- The final DPO loss function is:

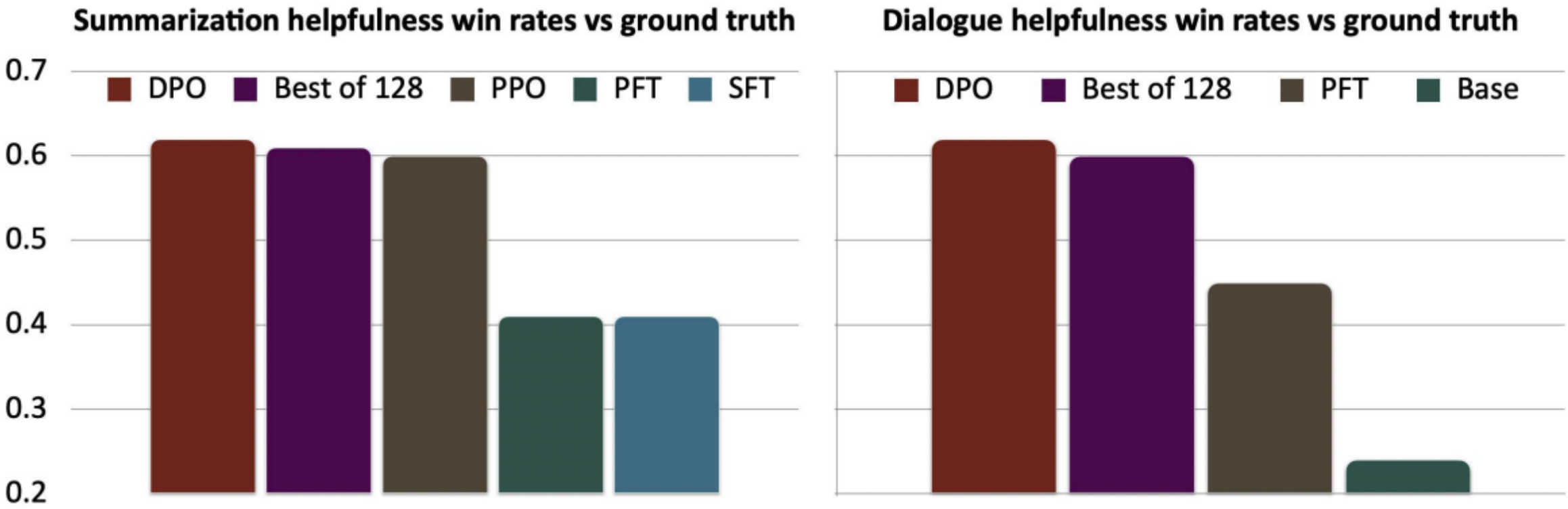
$$J_{RM}(\theta) = -E_{(x,y^w,y^l)\sim D}[\log \sigma(\beta \log \frac{p_\theta^{RL}(y^w|x)}{p^{PT}(y^w|x)} - \beta \log \frac{p_\theta^{RL}(y^l|x)}{p^{PT}(y^l|x)})]$$

We cancelled out both RM_ϕ and J^*

- Now we have a loss function that optimizes the language model parameters θ directly using preference data, in a fine-tuning way!

Rafailov et al (2023)

Direct Preference Optimization Works Well



Summary for Reinforcement Learning

- We want to optimize for human preferences
  - Instead of humans writing the answers or giving uncalibrated scores, we get humans to rank different LM generated answers.
- RLHF
  - Train an explicit reward model on comparison data to predict a score for a given completion.
  - Optimize the LM to maximize the predicted score (under KL-constraint)
  - Very effective when tuned well, computationally expensive and tricky to get right.
- DPO
  - Optimize LM parameters directly on preference data
  - Simple and effective, similar properties to RLHF, does not leverage online data.

Limitations of RL + Reward Modeling

- Human preferences are unreliable!
  - “Reward hacking” is a common problem in RL
  - Chatbots are rewarded to produce responses that seem authoritative and helpful, regardless of truth
  - This can result in making up facts + hallucinations
- Models of human preferences are even more unreliable!
- There is a real concern of AI mis(alignment)!

Factuality and Hallucination

- When you fine-tune a bag-of-words model on sentiment
  - You learn word meanings from the data itself.
- When you fine-tune an embedding-based model or BERT on sentiment
  - You still learn from the data,
  - The pre-training helps generalize.
- When a language model is prompted to do a task like sentiment,
  - You really don't see enough data points to "learn" much.
  - You're relying on the model's pre-training.

- What implications does this have for producing factual knowledge from LMs?

Factuality and Hallucination

- Language models model distributions over text, not facts. There's no guarantee that what they generate is factual:
  - Language models are trained on the web. Widely-popularized falsehoods may be reproduced in language models.
  - A language model may not be able to store all rare facts, and as a result moderate probability (uncertainty or lack of strong evidence) is assigned to several options.

There are many proposed solutions to factuality. How do we evaluate them? How do we check facts "explicitly"?

Hallucination

LLMs are prone to generate untruthful information that either conflicts with the existing source or cannot be verified by the available source. Even the most powerful LLMs such as ChatGPT face great challenges in migrating the hallucinations the generated texts. This issue can be partially alleviated by special approaches such as alignment tuning and tool utilization.

Factuality and Hallucination

There are still many limitations of large LMs (size, hallucination) that may not be solvable with RLHF!

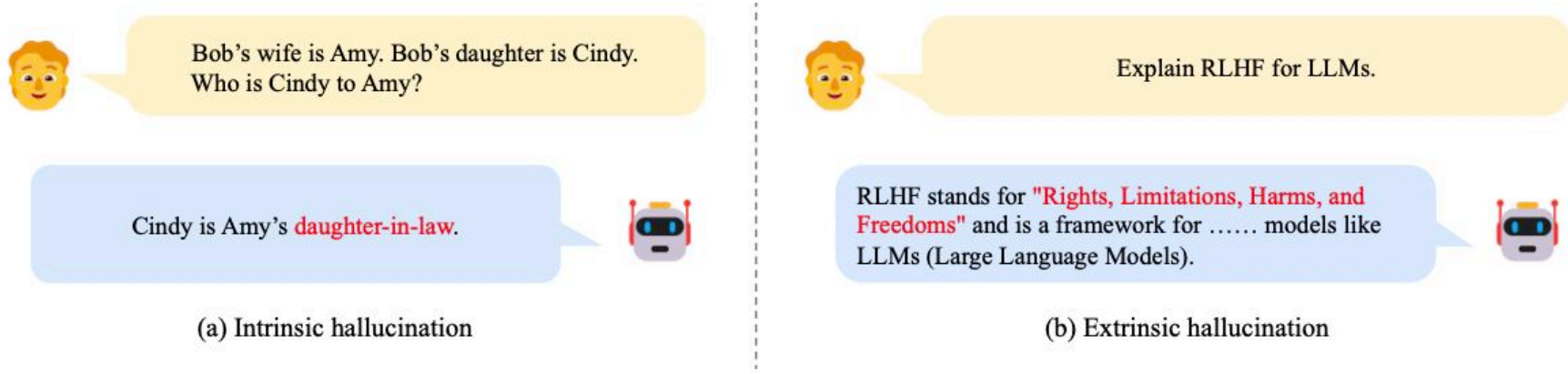


Fig. 14: Examples of intrinsic and extrinsic hallucination for a public LLM (access date: March 19, 2023). As an example of intrinsic hallucination, the LLM gives a conflicting judgment about the relationship between Cindy and Amy, which contradicts the input. For extrinsic hallucination, in this example, the LLM seems to have an incorrect understanding of the meaning of RLHF (reinforcement learning from human feedback), though it can correctly understand the meaning of LLMs (in this context).

Knowledge recency

- LLMs would encounter difficulties when solving tasks that require the latest knowledge beyond the training data.
- To tackle this issue, a straightforward approach is to regularly update LLMs with new data.
- However, it is very costly to fine-tune LLMs, and also likely to cause the catastrophic forgetting issue when incrementally training LLMs.

Therefore, it is necessary to develop efficient and effective approaches that can integrate new knowledge into existing LLMs, making them up-to-date.

Knowledge Recency

The parametric knowledge of LLMs is hard to be updated in a timely manner. Augmenting LLMs with external knowledge sources is a practical approach to tackling the issue. However, how to effectively update knowledge within LLMs remains an open research problem.

Evaluation metrics: ROUGE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

- Based on BLEU
- Not as good as human evaluation ('did this answer the user's question?')
- But much more convenient

Given a document D:

1. Have N humans produce a set of reference summaries of D
2. Run system, giving automatic summary X
3. What percentage of the N-grams from the reference summaries appear in X?

$$\text{ROUGE-N} = \frac{\sum_{S \in \text{References Summaries}} \sum_{\text{gram}_n \in S} \text{count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \text{References Summaries}} \sum_{\text{gram}_n \in S} \text{count}(\text{gram}_n)}$$

ROUGE: A Package for Automatic Evaluation of Summaries, Lin, 2004 http://www.aclweb.org/anthology/W04-1013

Evaluation metrics: ROUGE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

$$\text{ROUGE-N} = \frac{\sum_{S \in \text{References Summaries}} \sum_{\text{gram}_n \in S} \text{count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \text{References Summaries}} \sum_{\text{gram}_n \in S} \text{count}(\text{gram}_n)}$$

Like BLEU, it's based on n-gram overlap, differences:

- ROUGE has no brevity penalty
- Rouge is based on recall, while BLEU is based on precision
  - Precision is more important for MT (then add brevity penalty to fix under-translation)
  - Recall is more important for summarization (assuming you have a max length constraint)
  - However, often a F1 (combination of precision and recall) version of ROUGE is reported anyway.

ROUGE: A Package for Automatic Evaluation of Summaries, Lin, 2004 http://www.aclweb.org/anthology/W04-1013

Evaluation metrics: ROUGE

- BLEU is reported as a single number, which is combination of the precisions for n=1,2,3,4 n-grams.
- ROUGE scores are reported separately for each n-gram.
- The most commonly-reported ROUGE scores are:
  - ROUGE-1:* unigram overlap
  - ROUGE-2: bigram overlap
  - ROUGE-L: longest common subsequence overlap
- There is now a convenient Python implementation of ROUGE!

Evaluation of NLG

- There are not ideal evaluation for open-ended tasks.
  - Word overlap based metrics, such as BLEU, ROUGE, perperlexity.
  - No automatic metrics to adequately capture overall quality (i.e. a proxy for human quality judgment).
- More focused metrics to capture particular aspects of generated text:
  - Fluency (compute probability w.r.t. well-trained LM)
  - Correct style (prob w.r.t. LM trained on target corpus)
  - Diversity (rare word usage, uniqueness of n-grams)
  - Relevance to input (semantic similarity measures)
  - Simple things like length and repetition
  - Task-specific metrics e.g. compression rate for summarization
- Though these don't measure overall quality, they can help us track some important qualities that we care about.

Part 3: Learning from Verifiable Rewards

Rethinking the learning from human preferences

- Human preferences are expensive and noisy
  - “Alignment to humans” is intrinsically pluralistic
  - Can we have cheaper signals that are more verifiable?
- The LM are very good at “reward-hacking”
  - Achieve high scores on whatever you measure, while not improving much on real-world scenarios.
  - Can we set up fine-grained signals that avoid reward-hacking?
- A route becomes popular: RLVR (Reinforcement Learning from Verifiable Rewards)

RLVR: An Overview

- Instead of human preferences, compute a verifiable reward signal.
  - E.g., correctness of intermediate math steps, correctness of code snippets
- Use this reward signal to improve the policy (i.e., language model)
  - E.g., PPO or GRPO
  - Both PPO and GRPO are out of CS584's scope.

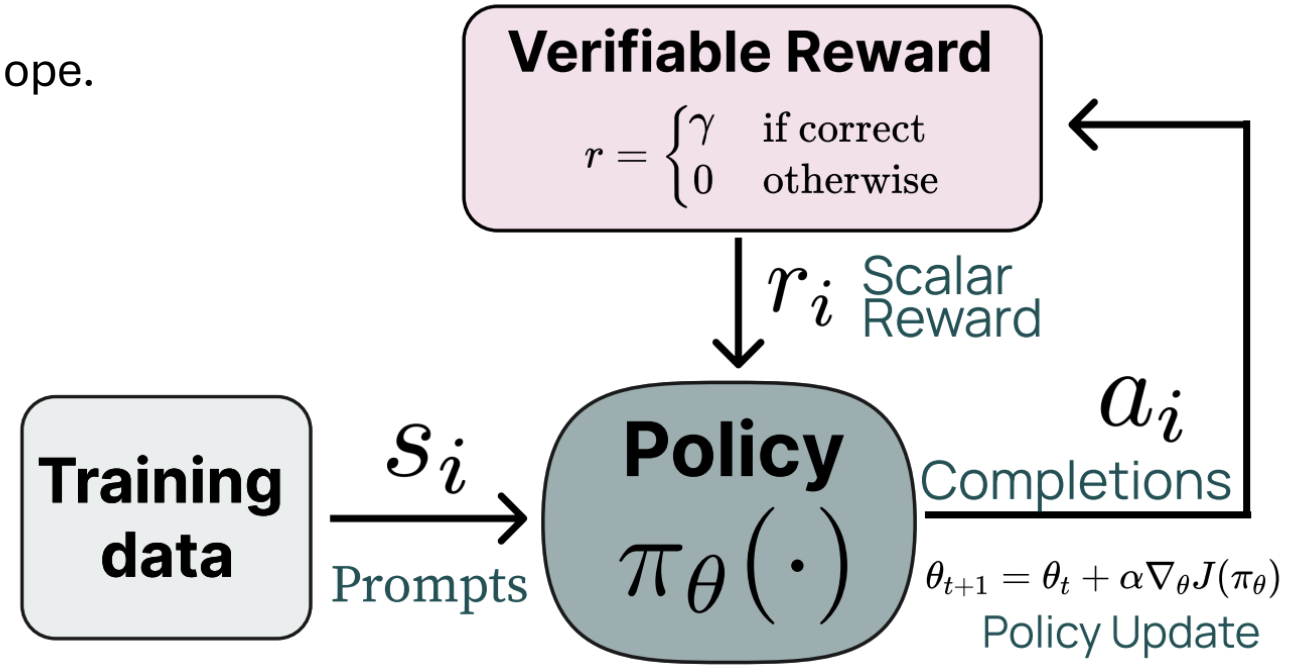


Figure from *"Tülu 3: Pushing Frontiers in Open Language Model Post-Training"*

Verifiable Rewards in DeepSeek-R1-Zero

- Accuracy Rewards
  - Math: the correctness of the answer within a box
  - Code: the execution correctness in a compiler
- Format Rewards
  - Enforce the reasoning process to appear between <think> and </think> tags.
- Combining the two rewards:

$$Reward_{rule} = Reward_{acc} + Reward_{format}$$

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. https://arxiv.org/pdf/2501.12948

RLVR surpasses human performance on AIME

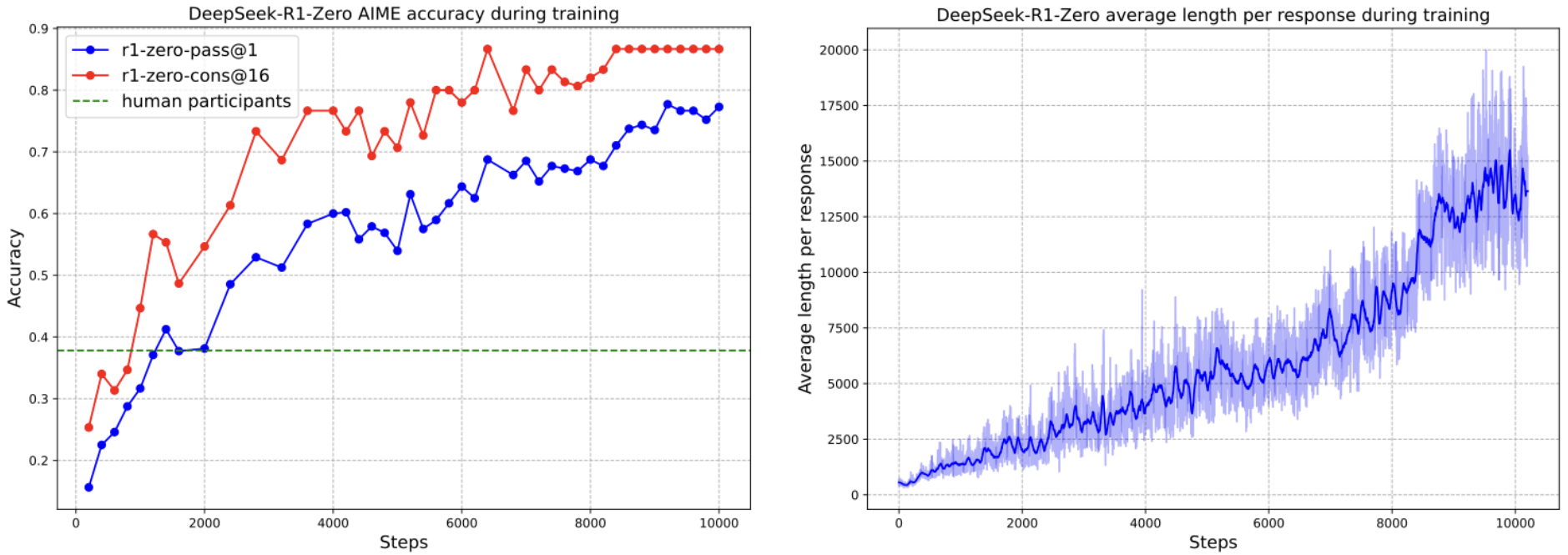


Figure 1 | (a) AIME accuracy of DeepSeek-R1-Zero during training. AIME takes a mathematical problem as input and a number as output, illustrated in Table 32. Pass@1 and Cons@16 are described in Supplementary D.1. The baseline is the average score achieved by human participants in the AIME competition. (b) The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time. Note that a training step refers to a single policy update operation.

RLVR elicits “aha moments”

Table 2 | An interesting “aha moment” of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to
Response: <think> To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$ $\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2$. Rearrange to isolate the inner square root term: $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$ $\ldots$ Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$ We started with the equation: $\sqrt{a - \sqrt{a + x}} = x$ First, let's square both sides: $a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$ Next, I could square both sides again, treating the equation: $\cdots$ $\ldots$

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. https://arxiv.org/pdf/2501.12948

Lesson from RLVR in DS-R1-Zero

“The self-evolution of DeepSeek-R1-Zero underscores the power and beauty of RL:
rather than explicitly teaching the model how to solve a problem,
we simply provide it with the right incentives,
and it autonomously develops advanced problem-solving strategies. “

“This serves as a reminder of the potential of RL to unlock higher levels of capabilities in LLMs, paving the way for more autonomous and adaptive models in the future.”

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. https://arxiv.org/pdf/2501.12948

RL from Verifiable Rewards vs Human Preferences

- These are two post-training paradigms.
- Difference: where does the reward come from.

- In either paradigm, many RL algorithms can apply.
  - E.g., PPO, GRPO, etc.
  - Note that DPO specifically applies to RLHF when there are fixed pairs of preference data ("offline learning").

Today’s Lecture: post-training techniques

- In-context learning
- Optimizing for human preferences
  - RLHF
  - DPO
- Learning verifiable rewards (RLVR)

Readings

1. Language Models are Few-Shot Learners
2. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
3. Finetuned Language Models Are Zero-Shot Learners
4. Learning to summarize from human feedback

Acknowledgement

The slides are adapted from Stanford's CS224n, and slides from Drs. Zining Zhu, Ping Wang, and Yue Ning.

Course Outcomes

1. Implement gradient descent (GD) and stochastic gradient descent (SGD) techniques for learning problems and understand the theory behind them.
2. Apply word2vec models in real-world text corpora.
3. Understand the neural networks models and backpropagation optimization. Implement neural network models in TensorFlow or others.
4. Understand dependency parsing, recurrent neural networks and their application in NLP.
5. Understand convolutional neural networks and their application in NLP.
6. Understand sequence to sequence models and attention in NLP deep neural networks.
7. Understand the recent advances in NLP, such as transformers, pre-training, fine-tuning, prompting, etc.